- `${!array[@]}`
- Here, an array is the name of a given any array variable.
- `[me@linuxbox ~]$ foo=([2]=a [4]=b [6]=c)`
- `[me@linuxbox ~]$ for i in "${foo[@]}"; do echo $i; done`
- a
- b
- c
- `[me@linuxbox ~]$ for i in "${!foo[@]}"; do echo $i; done`
- 2
- 4
- 6

Adding more elements in the array:

Shell provides a special += assignment operator that can automatically append the values at the end of an array. In the below example there is a given array foo with three elements and three new values assigned to it.

- `[me@linuxbox ~]$ foo=(a b c)`
- `[me@linuxbox ~]$ echo ${foo[@]}`
- a b c
- `[me@linuxbox ~]$ foo+=(d e f g h)`
- `[me@linuxbox ~]$ echo ${foo[@]}`
- a b c d e f g h

Sorting an Array:

You can also sort the values inside a column of the data. Though shell has no direct solution, with a little manipulation it can be done. Example:

- `#!/bin/bash`
- `# array-sort : Sort an array`
- `a=(f e g d c h b a)`
- `echo "Original array: ${a[@]}"`
- `a_sorted=($(for i in "${a[@]}"; do echo $i; done | sort))`
- `echo "Sorted array: ${a_sorted[@]}"`
- As the following executes the script results into;
- `[me@linuxbox ~]$ array-sort`
- `Original array: f e g d c h b a`
- `Sorted array: a b c d e f g h`
- Here the script first copies the content from the original array to a second array named (a_sorted) with a little change in the code.

Deleting an Array:

Unset command is called to delete an array. Here is an example of an array foo, first echo to check its presence and unset to delete it.